

CONTEXT

The course project accounts for **20% of your final grade** in **EECS 4312**. The **submission deadline is April 3, 2026**.

Please note a small adjustment to the project structure compared to what is stated in the course syllabus. To simplify the process, the **peer review phase will not take place for this project**, and the corresponding weight has been **reallocated to the project component**.

In this project, we work on real-world projects based on the concepts in Experimental Education and Hands-on Learning to enable you to apply your critical thinking and reflect on what you have learned. This is based on our strong belief that "one size does not fit all".

OBJECTIVE OF THIS DOCUMENT

Software requirements are typically documented using natural language. Understanding and retrieving this information is crucial for various stages of the software development lifecycle. This document serves as the project description for students enrolled in EECS 4312 during the Winter 2026 semester.

QUESTIONS

- For any questions, please attend the labs and lectures.
- If emailing questions, please anticipate at least one business day needed to receive a reply.

ABBREVIATIONS & DEFINITIONS

The terms in this document might be unfamiliar to you as a beginner; in case needed, please refer to [GitHub Glossary](#) and the [Machine Learning Glossary](#).

AMENDMENTS

- So far so good.

POLICIES

Your (submitted or un-submitted) solution to this assignment (which is not revealed to the public) remains the property of the EECS department. Do not distribute or share your

code in any public media (e.g., a non-private GitHub repository) in any way, shape, or form before you get permission from your instructors.

- You are required to work on your own for this lab. No group partners are allowed.
- When you submit your solution, you claim that it is solely your work. Therefore, it is considered a violation of academic integrity if you copy or share any parts of your code or documentation.
- When assessing your submission, the instructor and TA may examine your doc/code, and suspicious submissions will be reported to the department/faculty if necessary. We do not tolerate academic dishonesty, so please obey this policy strictly.
- Emailing your solutions to the instructor or TAs will not be acceptable.
- For any extension request, please refer to the course syllabus.

BACKGROUND

- Lab of week 6 and Lab of Week 11,
- Lecture Week 1 to Week 11.

DUE DATE

Make sure to submit ALL the files requested by April 3rd, 2025

WHAT TO SUBMIT

1- GitHub Repository (Primary Submission)

- Create a private repository named **SpecChain-StudentID** by forking from the repository:
https://github.com/EECS4312TeachingTeam/EECS4312_W26_SpecChain
- Add the **EECS4312TeachingTeam** account as a collaborator. Your repository must contain the following folders and files:

data/

- reviews_raw.jsonl
- reviews_clean.jsonl
- dataset_metadata.json
- review_groups_manual.json
- review_groups_hybrid.json

personas/

- personas_manual.json
- personas_auto.json
- personas_hybrid.json

spec/

- spec_manual.md
- spec_auto.md
- spec_hybrid.md

metrics/

- metrics_manual.json
- metrics_auto.json
- metrics_hybrid.json

README.md**reflection/**

- reflection.md

tests/

- tests_manual.json
- tests_auto.json
- tests_hybrid.json

Src/

- | | |
|--------------------------------|------------------------|
| • 00_validate_repo.py | • 05_personas_auto.py |
| • 01_collect_or_import.py | • 06_spec_generate.py |
| • 02_clean.py | • 07_tests_generate.py |
| • 03_manual_coding_template.py | • 08_metrics.py |
| • 04_personas_manual.py | • run_all.py |

2- eClass Submission (Cover File)

Submit a single PDF containing the name **SpecChain-YourLastName_studentID.pdf**, including:

- GitHub repository link
- App name
- Final dataset size
- Metrics summary table
- 1-page reflection summary
- Include your full name and student ID in the header.

HOW TO SUBMIT

1. Ensure all files are committed to GitHub before the deadline.
2. Confirm the notebook runs from top to bottom.
3. Add **EECS4312TeachingTeam** as collaborators.
4. Upload the required PDF to eClass.

Late commits after the deadline will not be graded.

FINAL CHECKLIST

- | | |
|---|---|
| ☐ 1- Repository structure complete | ☐ 6- Metrics computed |
| ☐ 2- Dataset cleaned and committed | ☐ 7- Notebook reproducible |
| ☐ 3- All persona files present | ☐ 8- Reflection included |
| ☐ 4- Specifications complete | ☐ 9- README updated |
| ☐ 5- Tests included | ☐ 10- All files committed |

THE PROJECT

Modern software systems are increasingly shaped by large volumes of informal natural language input such as user feedback, comments, and stakeholder descriptions. Software engineers must transform this unstructured information into structured, testable, and verifiable representations that guide development. In this project, you explore how different transformation pipelines affect the quality and reliability of that process. You will design and implement three pipelines that convert natural language input into:

- Structured personas
- Formalized specifications
- Validation tests
- Traceability links between artifacts

The goal is to understand how developer reasoning, automated generation, and hybrid workflows differ in producing structured software representations. This project integrates key concepts from requirements engineering, including:

- Stakeholder interpretation
- Abstraction and modeling
- Specification structuring
- Test design
- Traceability
- Verification and validation

You will also practice reproducible software engineering workflows using Python and GitHub.

TASK 1: FORK YOUR REPOSITORY AND INITIATE YOUR ENVIRONMENT

1. Clone the following repository by creating a fork:

https://github.com/EECS4312TeachingTeam/EECS4312_W26_SpecChain

2. Rename the repo to include your student ID:
EECS4312_W26_SpecChain_StudentID
3. Add **EECS4312TeachingTeam** as collaborators.

TASK 2: GATHERING REVIEWS

Pick your app as instructed in APPENDIX I of this document. Collect between **1,000 and 5,000 reviews** for the assigned application on the Google Play Store. Store the raw dataset in: `data/reviews_raw.jsonl`

Implement a cleaning script (`src/02_clean.py`) that removes duplicates, empty entries, and extremely short reviews. Remove punctuation, remove special characters and emojis, convert numbers to text, remove extra whitespace, convert all words to lowercase, remove stop words, and lemmatize the reviews. Save the cleaned dataset in: `data/reviews_clean.jsonl`

Create a metadata file. This file must document the app name, dataset size, collection method, and cleaning decisions. This metadata file should be named as: `data/dataset_metadata.json`

NOTES:

- 1- There is a limit on the number of reviews you can get from the app store. If your app has less than 1,000 (ish) reviews, that should be ok. If the app has more, gather as much as possible and add a note to your assignment, stating this limitation. Include how many reviews exists and how many you extracted.
- 2- You may use one of the existing web scrapers for the app stores (for example, [1](#), [2](#), [3](#), [4](#)) or write your own crawler for gathering the reviews.
- 3- You cannot change the set of apps (competitor or similar) compared to your Assignment.
- 4- *Good to know pre-processing is a standard task in machine learning! We Almost never run any automation on text without pre-processing it.*

TASK 3: BUILD THE MANUAL PIPELINE

In this task, you will manually transform the cleaned dataset into structured software representations. The goal is to *construct personas, derive specifications from them, generate validation tests from those specifications, and compute evaluation metrics*. All artifacts produced in this task must be based on the **cleaned dataset** (`data/reviews_clean.jsonl`).

TASK 3.1. Identify Review Groups: Open the cleaned dataset and read through the reviews to identify groups of similar feedback. Create **5 groups of reviews** that represent distinct types of users or user situations. Each group must include **at least 10 reviews**.

Use the template in `review_groups_manual.json` and save the groups in `data/review_groups_manual.json`. The review IDs should refer to the IDs in `data/reviews_clean.jsonl`.

TASK 3.2. Construct Personas from Review Groups: For each review group, create one persona that summarizes the group's common goals, pain points, and context. Save personas using the provided template in: `personas/personas_manual.json`. Each persona must clearly reference the **review group from which it was derived**, following the provided template and example.

Step 3.3. Generate Specifications from Personas: Using the personas, derive a structured specification describing what the system should support. Use the template in `spec/spec_manual.md` to write the specification. Each requirement must include: a unique requirement ID, a clear description, the persona it originated from, and acceptance criteria. Write at least **10 requirements**.

Step 3.4. Generate Validation Tests: Create validation tests based on the specifications. Save the tests using the template in: `tests/tests_manual.json`. **Each requirement must have at least two test scenarios.**

Step 3.5. Compute Metrics: Compute metrics for the manual pipeline. Save results in: `metrics/metrics_manual.json` by computing the requested metrics in the template. Metrics to compute:

- *Dataset size:* The total number of cleaned user reviews available in `data/reviews_clean.jsonl` after preprocessing.
- *Persona count:* The number of personas created from manually identified review groups in `personas/personas_manual.json`.
- *Requirements count:* The total number of software requirements documented in the manual specification file `spec/spec_manual.md`.
- *Tests count:* The number of validation test scenarios defined in `tests/tests_manual.json`.
- *Traceability links:* The number of explicit traceable relationships between artifacts in the pipeline.
- *Review coverage ratio:* The ratio (0-1) of reviews that have been covered by the personas generated.
- *Traceability ratio:* The proportion of requirements that can be traced back to a persona through an explicit reference in the specification.
- *Testability rate:* The proportion of requirements that have at least one associated validation test scenario in `tests/tests_manual.json`.
- *Ambiguity ratio:* The proportion of requirements whose descriptions or acceptance criteria contain ambiguous or non-measurable language (e.g., vague terms such as fast, easy, better, user-friendly, etc.) that lead to multiple interpretations during testing.

Deliverables for TASK 3: After completing Task 3, your repository must contain:

- data/review_groups_manual.json
- personas/personas_manual.json
- spec/spec_manual.md
- tests/tests_manual.feature
- metrics/metrics_manual.json

TASK 4: BUILD THE AUTOMATED PIPELINE

In this task, you will construct an automated pipeline that generates personas, specifications, and validation tests from the dataset using Python scripts and LLM prompting. All artifacts in this task must be generated automatically using the cleaned dataset: data/reviews_clean.jsonl

The automated pipeline must produce the same types of artifacts as the manual pipeline so that the outputs can be compared.

Step 4.1. Group Reviews Automatically: This script in src/05_personas_auto.py must load the cleaned dataset and automatically group similar reviews using a clustering or grouping approach. Acceptable approaches include semantic similarity grouping, embedding clustering, or LLM-based grouping, or a combination of them, in any way that generates meaningful results. You may only use the Groq API with meta-llama/llama-4-scout-17b-16e-instruct as instructed in your Prompt engineering lab of EECS 4312.

The script should produce review groups similar (not identical) to those created manually in TASK 3. Save the results in: data/review_groups_auto.json. Each group must contain at least several related reviews and clearly represent a common theme. Save the prompt you used for this automation in prompts/prompt_auto.json.

Step 4.2. Generate Personas Automatically: Use the review groups from Step 4.1 to automatically create personas. The script src/05_personas_auto.py must generate structured persona objects using an LLM prompt or automated summarization.

As for the LLM, you will be using Groq API with meta-llama/llama-4-scout-17b-16e-instruct as you instructed in Prompt Engineering Lab. Save the output in: personas/personas_auto.json. Each persona must reference the review group it was derived from and use a template similar to that in personas/personas_manual.json.

Step 4.3. Generate Specifications Automatically: Create or reuse a script in src/06_spec_generate.py. This script must read the automated personas and generate structured system requirements. Save the generated specification in: spec/spec_auto.md.

Each requirement must include: *a unique requirement ID, a description of system behavior, the persona that motivated the requirement, traceability to the review group, and acceptance criteria.* The format must follow the template is provided in the file.

Step 4.4. Generate Validation Tests Automatically: Create validation tests based on the automatically generated specifications. Each requirement must include at least one validation scenario that describes how it could be verified. Implement this step using the script: `src/07_tests_generate.py`

This script must read the requirements from: `spec/spec_auto.md` and generate structured test scenarios. Save the generated tests in: `tests/tests_auto.json`. Each test scenario must contain: *a unique test ID, the requirement it validates, a short scenario description, a list of steps* describing how the test would be executed, and the *expected outcome*. A template is provided in the file, and the format must be followed.

Each requirement in the specification must be linked to at least **one test scenario**. The script should ensure that every test references a *requirement ID*, that the *steps* are written clearly, and that the *expected result* reflects the requirement being validated. The automated pipeline must produce: `tests/tests_auto.json`

Step 4.5. Compute Metrics for the Automated Pipeline: Use the script `src/08_metrics.py`. Compute the same metrics used for the manual pipeline. Save the results in: `metrics/metrics_auto.json`. Metrics must be computed from the artifacts generated in this pipeline. After completing the automated pipeline, your repository must contain the following files:

- `data/review_groups_auto.json`
- `personas/personas_auto.json`
- `spec/spec_auto.md`
- `tests/tests_auto.json`
- `metrics/metrics_auto.json`
- `prompts/prompt_auto.json`

All files must be committed before proceeding to the hybrid pipeline.

TASK 5: BUILD THE HYBRID PIPELINE

The purpose of this task is to create a third pipeline that combines the speed of automation with human judgment. You must not create the hybrid artifacts from scratch. Instead, you must begin with the automated outputs and revise them. All files in this task must be based on the automated pipeline outputs already created in TASK 4. In this task, use the automated pipeline's outputs as a starting point and refine them manually.

Step 5.1. Refine the Automated Review Groups: Open the automated review grouping file: `data/review_groups_auto.json`. Read each group carefully. Check whether the theme matches the included reviews, whether unrelated reviews have been grouped together, and whether any important type of feedback is missing. Create a revised version of the grouping file and save it as: `data/review_groups_hybrid.json`.

Keep the same overall structure, but update the group theme, example reviews, or review membership where necessary. An example template is provided in the file for your reference. The notes field should briefly explain what was corrected.

Step 5.2. Refine the Automated Personas: Open the automated persona file: `personas/personas_auto.json`. Review each persona and correct any vague descriptions, missing context, unsupported claims, or inconsistent constraints. Do not invent details that are not supported by the review group. **Keep each persona grounded in the grouped review evidence.** Save the revised personas as: `personas/personas_hybrid.json`. Every hybrid persona must preserve a clear link to the review group it came from. An example template is provided in the file.

Step 5.3. Refine the Specifications: Open the automated specification file: `spec/spec_auto.md`. Review each requirement. Rewrite any requirement that is vague, too broad, unsupported by the persona, or difficult to test. Make sure each requirement is clearly motivated by the hybrid persona and traceable to the hybrid review group. Save the revised specification as: `spec/spec_hybrid.md`. The hybrid specification should still contain at least 10 requirements. Follow the template provided in the file.

Step 5.4. Refine the Validation Tests: Open the automated test file: `tests/tests_auto.json`. Review the tests and revise any scenario that is unclear, disconnected from the requirement, or missing an expected result. Save the revised tests as: `tests/tests_hybrid.json`. Every test must reference a hybrid requirement ID.

Step 5.5. Compute Hybrid Metrics: Use the same metric script from the earlier tasks: `src/08_metrics.py`. Compute the hybrid pipeline metrics and save them as: `metrics/metrics_hybrid.json`. Commit all hybrid pipeline files after completing this task.

Deliverables for Task 5: After completing Task 5, your repository must contain:

- `data/review_groups_hybrid.json`
- `personas/personas_hybrid.json`
- `spec/spec_hybrid.md`
- `tests/tests_hybrid.json`
- `metrics/metrics_hybrid.json`

TASK 6: COMPUTE AND COMPARE METRICS ACROSS PIPELINES

In this task, compute the same set of metrics for the manual, automated, and hybrid pipelines and prepare them for comparison. Use the script: `src/08_metrics.py`.

This script must read the outputs of each pipeline and calculate the required metrics. At minimum, compute the number of personas, the number of requirements, the number of

tests, review coverage, traceability ratio, and testability rate. Store one metrics file for each pipeline in the metrics/ folder:

- metrics/metrics_manual.json
- metrics/metrics_auto.json
- metrics/metrics_hybrid.json

This summary file will make it easier to compare the three pipelines and discuss differences in the final reflection: metrics/metrics_summary.json. An example template of the components of this file is provided for your reference.

Deliverables for Task 6: After completing Task 6, your repository must contain the following list, which the last two items are directly produced in this step:

- metrics/metrics_manual.json
- metrics/metrics_auto.json
- metrics/metrics_hybrid.json
- metrics/metrics_summary.json

TASK 7: ENSURE REPRODUCIBILITY

In this task, make sure your repository can be executed and understood by another person. Open the src/ folder and confirm that all required scripts are present and clearly commented. Then implement: src/run_all.py

This script must execute the automated workflow from start to finish. It should load the raw dataset, clean it, generate automated review groups and personas, create specifications, generate tests, and compute metrics. **The script does not need to automate manual or hybrid thinking steps**, but it must automate all steps intended to be generated programmatically.

Add comments in run_all.py describing the order of execution and the files produced at each stage. Run the script using: python src/run_all.py. Fix any errors before submission. Then implement a repository validation script: src/00_validate_repo.py

This script must check whether all required folders and required files exist. It should print a clear message indicating whether the repository structure is complete:

```
Checking repository structure...
data/reviews_clean.jsonl found
personas/personas_manual.json found
spec/spec_auto.md found
tests/tests_hybrid.json found
Repository validation complete
```

Deliverables for Task 7: After completing Task 7, your repository must contain:

- `src/run_all.py`
- `src/00_validate_repo.py`

Your repository must also run without missing-file errors.

TASK 8: FINAL DOCUMENTATION AND REFLECTION

In this task, create the final documentation files needed to explain and submit your project.

First, update the root `README.md` file. This file must explain the application studied, the size of the final dataset, the folder structure, and the exact commands required to run the scripts. It must be possible for a TA to open the `README` and understand how the repository is organized and how to reproduce the automated pipeline.

Next, create the reflection file: `reflection/reflection.md`. In this file, explain how the manual, automated, and hybrid pipelines differed. Discuss which pipeline produced the clearest personas, which produced the most useful requirements, which had the strongest traceability, and what types of problems appeared in the automated outputs. Keep the reflection concise but specific.

Deliverables for TASK 8: After completing TASK 8, your repository must contain:

- `README.md`
- `reflection/reflection.md`

Push all files and commits before the deadline.

APPENDIX 1: ASSIGNED APPS

The first column is the first letter of your family name, based on that, your assigned app is listed in front:

First Letter of your Family Name	Link to the Google Play page of your assigned app
A, B, C	Head space (link)
D, E	Mind Doc (link)
F, G, H	Wysa (link)
I, J, K	Calm (link)
L, M, N	Medito (link)
O, P, Q	Head space (link)
R, S, T	Mind Doc (link)
U, V, W	Wysa (link)
X, Y, Z	Calm (link)